

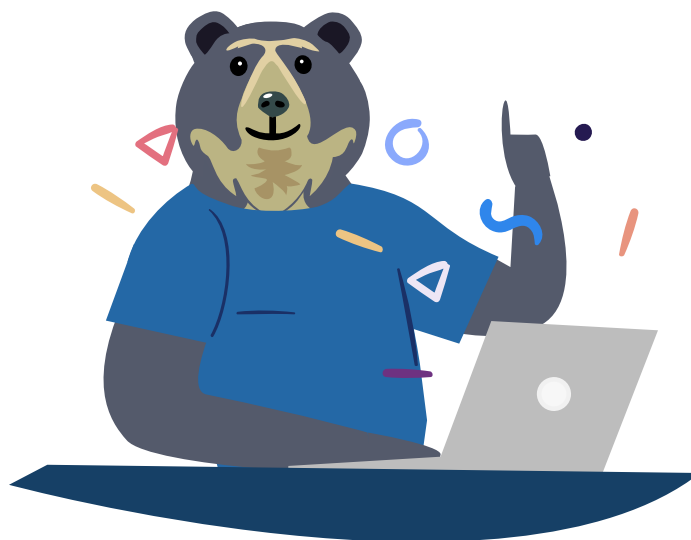


Bienvenido a la Prueba Práctica

Este concurso está habilitado a todos los postulantes que pasaron a la siguiente fase en el proceso de selección del Bootcamp DEV v8 de Digital Harbor

Requisitos y directrices

- Los tópicos que se evaluarán son: operadores matemáticos, operadores lógicos, condicionales, bucles, cadenas, arreglos
- Se podrá enviar los resultados en los lenguajes: C++, Java, NodeJS, Python 3
- La prueba constará de 5 ejercicios
- La prueba estará habilitada entre las 07:30 y 09:30 horas
- Deben permanecer conectados durante toda la reunión y mantener cámaras encendidas



Nota

- Cualquier duda durante el test, por favor puedes escribir al Trainer encargado en Teams.



Teleferico Paceño

Time Limit: 0.2 seconds **Memory Limit:** 256 MB

Type: Standard **Mode:** Partial

Description

En la ciudad de La Paz, el sistema de teleféricos realiza un censo diario de pasajeros. Cada línea del teleférico (Roja, Amarilla, Verde, Azul, etc) registra el carnet de identidad de los pasajeros que utilizaron esa línea durante el día.

El ministerio de Obras publicas quiere saber:

- Cuantos pasajeros unicos usaron el sistema ese día.
- Que línea tuvo mas pasajeros distintos
- Cuantos pasajeros usaron mas de una línea

Input

La primera línea contiene un entero N ($1 \leq N \leq 10^4$) número de registros.

Las siguientes N líneas tienen el formato `<línea> <ci>`. Donde `<línea>` es una cadena sin espacios (ej. Roja, Verde, Amarilla) y `<ci>` es un número entero positivo que representa el carnet de identidad

Output

En la salida se deben imprimir 3 líneas

- Cantidad total de pasajeros unicos.
- Nombre de la línea con mas pasajeros distintos.
- Cantidad de pasajeros que usaron mas de una línea

Input sample

```
7
Roja 12345
Verde 99999
Roja 88888
Amarilla 12345
Verde 12345
Azul 77777
Roja 99999
```

```
5
Roja 100
Verde 200
Amarilla 300
```

Output sample

```
4
Roja
2
```

```
5
Roja
0
```



Azul 400

Plateada 500



Evaluacion de Proyectos en DH

Time Limit: 0.2 seconds **Memory Limit:** 256 MB

Type: Standard **Mode:** Partial

Description

La empresa Digital Harbor se encuentra evaluando una serie de proyectos tecnológicos antes de aprobar un despliegue importante en producción.

Cada proyecto i tiene asociado:

- un nivel de complejidad x_i
- un incremento de capacidad y_i que se obtiene al completar exitosamente el proyecto

Reglas:

- Los proyectos pueden evaluarse en cualquier orden.
- Si en algun momento la capacidad actual no es estrictamente mayor que la complejidad del proyecto ($S \leq x_i$) el proyecto no puede completarse y el proceso se detiene.
- Si la capacidad actual si es mayor ($S > x_i$), el proyecto se completa exitosamente y la capacidad de digital harbor aumenta en y_i

Objetivo:

Determinar si Digital Harbor puede completar todos los proyectos sin fallar en ninguna evaluacion y avanzar al siguiente despliegue.

Input

La primera linea contiene dos enteros **S** y **n** ($1 \leq S \leq 10^4$, $1 \leq n \leq 10^3$)

Las siguientes **n** lineas contienen dos enteros x_i y y_i ($1 \leq x_i \leq 10^4$, $0 \leq y_i \leq 10^4$)

Output

Imprimir "YES" (sin comillas) si Digital Harbor puede completar todos los proyectos

Imprimir "No" (sin comillas) si no es posible

Input sample

```
2 2
1 99
100 0
```

```
10 1
100 100
```

Output sample

```
YES
```

```
NO
```



Sistema de combate

Time Limit: 0.2 seconds **Memory Limit:** 256 MB

Type: Standard **Mode:** Partial

Description

Una empresa local está a punto de lanzar un nuevo videojuego al mercado y has sido contratado para realizar pruebas automatizadas, a fin de identificar si se requiere balancear sus mecánicas. El videojuego es de un solo jugador y debes evaluar el sistema de combate. Para realizar tus pruebas, considera que:

- El combate es por turnos, en los que el jugador se enfrenta a N enemigos.
- El jugador ataca primero y todos los enemigos que sigan con vida lo atacan después.
- Los enemigos pueden aplicar un stun al jugador cada cierta cantidad de turnos, tras lo cual, en el siguiente turno, el jugador no puede atacar pero los enemigos sí.
- Si varios enemigos aplican stun en el mismo turno, el jugador solo es afectado por 1 turno y los enemigos no pueden guardar el stun.

El objetivo de la prueba es determinar la menor cantidad de turnos para derrotar a todos los enemigos, o decidir que es imposible ganar.

Input

La primera línea contiene la cantidad de pruebas a realizar, T , donde $1 \leq T \leq 10$.

Cada prueba esta formada por

- 1 línea con la cantidad de enemigos a enfrentar, N
- 1 línea con la salud H y el daño A del jugador
- N líneas con la salud, el daño y el numero de turnos para stun de los enemigos ($h[i]$, $a[i]$, $s[i]$).

Para cada prueba, se sabe que:

- $1 \leq N \leq 500$
- $1 \leq H \leq 10^6$
- $1 \leq A \leq 100$
- $1 \leq h[i] \leq 10^5$
- $1 \leq a[i] \leq 10^4$
- $2 \leq s[i] \leq 10^5$

Output

Una línea por prueba, con la cantidad mínima de turnos para ganar, o -1 si es imposible hacerlo.

Input sample

```
2
2
100 10
30 5 100
```

Output sample

```
5
10
```



20 3 100

3

120 10

30 6 3

50 8 100

10 2 2

1

2

100 10

30 5 2

20 3 100

6



Sistema de turnos en Banco

Time Limit: 0.3 seconds **Memory Limit:** 256 MB

Type: Standard **Mode:** Partial

Description

Es la primera semana de mes y se pagan bonos y sueldos, trabajas en una sucursal pequeña de un banco importante en una zona muy demandada, actualmente la sucursal cuenta con 2 cajeros y la atención por turnos es muy poco eficiente por lo cual el gerente te pide que reprogrames el sistema de turnos y calcules en qué minuto sale el último cliente de una tanda de clientes.

La sucursal tiene una sola fila de espera y atiende 3 tipos de clientes, además por reglas del banco siempre se cuenta con un cajero experimentado y un cajero sin mucha experiencia

Reglas:

clientes

- J = jubilado: Preferencia alta, no puede esperar mucho.
- C = cliente bancario: preferencia media, retiran o depositan dinero.
- S = pago de servicios: preferencia baja, solo pagan servicios como la luz o agua

Cajeros

- Caja 1 (Cajero experimentado): Atiende a todos y tarda 5 min X cliente
 - Atiende primero a un cliente tipo J y si no hay ninguno pasa a un cliente tipo C.
 - Solo atiende clientes tipo S si llevan mas de 10 min en la fila
- Caja 2 (Cajero nuevo): Solo atiende pago de servicios, tarda 2min X cliente.
 - Solo atiende clientes tipo S, no atiende C ni J.
 -

Se necesita un algoritmo que reciba una lista de clientes que incluya el minuto de llegada y el tipo de cliente respectivamente y que calcule en que minuto sale el ultimo cliente.

Input

La primera línea contiene un entero N ($1 \leq N \leq 1000$), el número de clientes que llegarán hoy.

Las siguientes N líneas tienen el formato `<minuto> <tipo>`.

- `<minuto>`: Un número entero positivo ($0 \leq N \leq 1000$) que indica el minuto de llegada. Los datos vienen ordenados por tiempo de llegada.
- `<tipo>`: Un carácter que representa el tipo de cliente:
 - J: Jubilado
 - C: Cliente Bancario
 - S: Pago de Servicios

Output

Un único número entero que representa el **minuto final** en el que se desocupa el último cajero (cuando el último cliente termina de ser atendido).



Input sample

5
0 S
0 C
1 J
2 S
15 S

6
0 S
0 S
1 S
50 C
51 J
52 S

Output sample

17

60



Torres de lego

Time Limit: 0.5 seconds **Memory Limit:** 256 MB

Type: Standard **Mode:** Partial

Description

Estás construyendo torres con bloques de Lego colocados uno encima de otro. Cada bloque tiene un color (un número del 1 al 10) y una altura positiva. El objetivo es determinar si la torre es **Valida** o **Invalida** de acuerdo con las siguientes reglas:

Las reglas son las siguientes:

1. Las alturas de los bloques deben aumentar estrictamente de abajo hacia arriba.
2. No pueden existir dos bloques consecutivos con el mismo color.
3. Si un bloque es de **color rojo (color = 1)**, debe estar ubicado en una posición par (segunda, cuarta, sexta, etc.).
4. La suma total de las alturas de todos los bloques debe ser al menos *Hmin*.
5. Si la **altura de un bloque es un número feo**, entonces su **color debe ser múltiplo de 2 o de 5, excepto si el bloque es rojo**.

Nota: Un número feo es aquel cuyos únicos factores primos son **2, 3 y 5**.

Tu tarea es leer la torre y determinar si es **Valida** o **Invalida**

Input

- La primera línea contiene dos enteros **N** y **Hmin**.
- La segunda línea contiene $2*N$ enteros:

`c1 h1 c2 h2 . . . cN hN`

Restricciones:

- $1 \leq N \leq 100$
- $1 \leq c_i \leq 10$
- $1 \leq h_i \leq 100$
- $1 \leq Hmin \leq 10^4$

Output

Imprimir "**Valida**" (sin comillas) si todos los bloques cumplen con las reglas o "**Invalida**" si alguna regla se rompe.

Input sample

```
5 15
2 2 3 3 6 4 7 6 8 9
```

```
3 6
2 2 4 3 5 4
```

Output sample

Invalida

Valida



8 30

2 1 4 2 6 3 5 4 8 6 10 8 12 9 14 10

Valida

